

Anexo - Herramientas externas y tecnologías utilizadas



”Porque el club es de los socios, y la gestión es de SocioUnido”

Plataforma inteligente que centraliza la gestión administrativa de los clubes, optimizando el uso de espacios compartidos, previniendo la morosidad y asegurando los accesos al estadio.

Alumno	Correo electrónico	Padrón
Ascencio, Felipe Santino	fascencio@fi.uba.ar	110675
Ghosn, Lautaro Gabriel	lghosn@fi.uba.ar	106998
Guerrero, Martín	mguerrero@fi.uba.ar	107774
Zielonka, Axel	azielonka@fi.uba.ar	110310

Jefe de Cátedra Mg. Ing. Carlos Fontela cfontela@fi.uba.ar

Tutor Mg. Ing. Gabriel Piñeiro gpineiro@fi.uba.ar

Cotutor Ing. Agustín Perrotta aperrotta@fi.uba.ar

Índice

1. Anexo - Herramientas externas y tecnologías utilizadas	3
1.1. Lenguajes de programación y estilos	3
1.2. Frameworks, librerías y arquitectura	3
1.3. Bases de datos, caché y despliegue	4
1.4. Herramientas de gestión, análisis de código y diseño	4
1.5. Declaración de uso de Inteligencia Artificial Generativa (IAG)	5

1. Anexo - Herramientas externas y tecnologías utilizadas

El presente anexo detalla el *stack* tecnológico y las herramientas externas implementadas a lo largo de todas las fases del Trabajo Profesional. De acuerdo con los lineamientos de ética y rigor académico, se justifica la elección de cada componente basándose en criterios técnicos, económicos (priorizando la gratuidad y el acceso libre u *open-source*) y su estandarización en la industria actual del desarrollo de *software*.

Asimismo, se incluye la declaración de responsabilidad respecto al uso de Inteligencia Artificial Generativa (IAG) por parte de cada integrante del equipo.

Nota: Es importante destacar que las integraciones con servicios externos (Mercado Pago, Telegram, etc.) se detallan en profundidad en el Anexo específico de “APIs de Terceros”.

1.1. Lenguajes de programación y estilos

- **Python:** Lenguaje principal del *back-end*. Se eligió por su versatilidad, su ecosistema inigualable para el análisis de datos (vital para el microservicio de analíticas) y su velocidad para desarrollar reglas de negocio complejas de forma mantenible.
- **JavaScript / JSX:** Utilizado para el *front-end*. Se seleccionó por ser el estándar absoluto de la web moderna, permitiendo agilidad en la codificación combinando sintaxis moderna con CSS.
- **Go (Golang):** Lenguaje subyacente del *API Gateway* (KrakenD). Su uso garantiza capacidades de concurrencia masiva, permitiendo manejar miles de peticiones por segundo con un consumo de recursos ínfimo.
- **HTML y CSS Modular:** Se utilizó CSS tradicional apoyado en variables (*tokens*) para facilitar la tematización de “marca blanca”, requerida para adaptar la identidad visual a distintos clubes sin reescribir lógica.
- **LaTeX:** Seleccionado para la redacción de la documentación académica final debido a su superioridad tipográfica y su estándar innegable en la publicación de documentos formales de ingeniería.
- **Ruby:** Utilizado subyacentemente para la generación de las páginas estáticas documentales mediante *JustTheDocs*.

1.2. Frameworks, librerías y arquitectura

- **FastAPI:** *Framework* web de Python seleccionado por su altísimo rendimiento asíncrono. Permitió estructurar múltiples controladores de forma limpia y autogenerar la documentación interactiva (Swagger/OpenAPI) fundamental para la interconexión de servicios.
- **React + Vite:** Elegidos para el *front-end* (plataforma administrativa y PWA) por su ecosistema maduro basado en componentes reutilizables. Vite reemplazó a Webpack por su velocidad de compilación ultrarrápida (HMR), optimizando los tiempos de desarrollo.
- **KrakenD:** Seleccionado como *API Gateway* por ser puramente declarativo (*stateless*). Permitió centralizar la seguridad y enrutar las peticiones hacia los microservicios escalando linealmente sin generar cuellos de botella en bases de datos.
- **SQLAlchemy y Alembic:** Estándar de la industria en Python para el modelado relacional masivo (ORM) y la ejecución controlada de migraciones sobre los esquemas de bases de datos de forma segura e incremental.
- **Pytest:** Adoptado como marco de pruebas unitarias y asíncronas por su ecosistema y el uso de *fixtures*, garantizando que las interacciones entre los múltiples submódulos del club no generen efectos secundarios indeseados antes de los despliegues.

1.3. Bases de datos, caché y despliegue

- **Neon (PostgreSQL):** Seleccionado como Sistema de Gestión de Bases de Datos (SGBD) relacional *serverless* en la nube. Se priorizó por su compatibilidad nativa con PostgreSQL, su separación de cómputo y almacenamiento, y su generosa capa gratuita ideal para el desarrollo de proyectos universitarios sin incurrir en costos de alojamiento.
- **Redis:** Utilizado como agente de mensajes (*message broker*) y caché temporal. Fundamental en el ecosistema analítico, permitió encolar y procesar eventos de manera asíncrona, desacoplando cargas pesadas del flujo principal de la aplicación.
- **Firebase:** Integrado como plataforma de servicios en la nube para resolver dos necesidades críticas de manera ágil. Por un lado, mediante *Firebase Authentication* se delegó la gestión del ciclo de vida de las contraseñas, la recuperación de cuentas y la emisión de tokens seguros, lo cual exime al sistema de almacenar contraseñas en texto plano y eleva significativamente el nivel de seguridad de la plataforma. Por otro lado, se implementó *Firebase Cloud Messaging* para proveer notificaciones *push* en tiempo real a los dispositivos de los socios.
- **Docker y Docker Compose:** Herramientas de contenerización indispensables en nuestra arquitectura. Nos permiten aislar cada microservicio de forma independiente y garantizar una paridad técnica exacta entre los entornos de desarrollo, *staging* y producción.
- **Make:** Utilizado para la orquestación y automatización mediante *Makefiles*, simplificando la ejecución rápida de contenedores y rutinas de *testing* desde la línea de comandos.
- **Vercel y Render:** Plataformas de computación en la nube (*Cloud Computing*) seleccionadas para el *hosting*. Vercel fue empleado por su optimización superior y despliegue continuo nativo para *front-ends* basados en Vite/React. Por su parte, Render fue utilizado para la ejecución ininterrumpida de los contenedores Docker del *back-end*. Ambas se eligieron por su robusta oferta de capa gratuita e integración perfecta con el flujo de CI/CD.

1.4. Herramientas de gestión, análisis de código y diseño

- **Git y GitHub:** Control de versiones y alojamiento de repositorios seleccionados por ser el estándar predominante de la industria. Su elección brindó control absoluto sobre los flujos de colaboración, gestión de revisiones cruzadas (*Pull Requests*) e integración con múltiples herramientas de terceros.
- **GitHub Actions:** Motor integrado para flujos de Integración y Despliegue Continuo (CI/CD). Fue preferido sobre opciones como Jenkins o Travis CI por la nula configuración de servidores externos y su ejecución automatizada sobre la propia infraestructura de GitHub.
- **SonarCloud y OpenClaw:** Plataformas de análisis de código estático (SAST) y aseguramiento de calidad (QA). Se utilizaron como auditores imparciales para garantizar la seguridad del código, evaluar la cobertura de *tests*, minimizar la deuda técnica y prevenir la exposición accidental de información crítica antes de llegar a producción.
- **Dependabot:** Herramienta nativa de GitHub implementada para la protección proactiva de dependencias en el ecosistema. Se eligió por su capacidad de escanear ininterrumpidamente los repositorios en busca de vulnerabilidades conocidas (CVEs) en librerías de terceros y generar automáticamente *Pull Requests* con las actualizaciones de seguridad, reduciendo drásticamente el tiempo de remediación.
- **Figma:** Empleado para la construcción interactiva de prototipos y *wireframes* (UX/UI). Elegido por su interfaz colaborativa en tiempo real, permitiendo previsualizar y afinar la experiencia del usuario final minimizando refactorizaciones durante la programación web.
- **JustTheDocs:** Tema estructural basado en Jekyll. Adoptado para conformar portales web de documentación técnica sumamente veloces, navegables y públicos que centralizan la información sobre implementaciones, *Scrum*, hojas de ruta y bitácoras semanales del proyecto.

1.5. Declaración de uso de Inteligencia Artificial Generativa (IAG)

En cumplimiento estricto de las directivas y consideraciones de ética profesional de la Facultad de Ingeniería de la UBA, este apartado declara el empleo de plataformas de Inteligencia Artificial Generativa (IAG) e infraestructuras de Grandes Modelos de Lenguaje (LLMs) durante el ciclo de vida del Trabajo Profesional.

Se deja asentada constancia explícita de que estas herramientas operaron exclusivamente bajo roles de asistencia técnica (resolución de *debugging* complejo, consultas semánticas, aceleración de tareas repetitivas [*boilerplate*] y soporte en la organización documental de *LaTeX*). **Todo el código generado fue sometido a revisiones cruzadas severas; cada estudiante del equipo se responsabiliza, asimila en profundidad y garantiza la auditoría técnica de los desarrollos aplicados**, asegurando su idoneidad para ser defendidos durante las instancias de evaluación.

El desglose del entorno informático e IAG reportado de manera individual por los integrantes es el siguiente:

Ascencio, Felipe Santino

Entorno de desarrollo: VS Code.

Modelos de asistencia IAG: Gemini (Google).

Aplicación principal: Empleada de forma continua para agilizar y optimizar los tiempos de desarrollo. Sus casos de uso abarcaron la optimización ortográfica de la documentación técnica, la diagramación de formatos estructurados de LaTeX y la asistencia como motor de búsqueda avanzado para esquemas de aseguramiento de calidad (QA). Adicionalmente, Gemini se integró como el motor LLM subyacente para el funcionamiento del agente de QA automatizado (OpenClaw), habiendo liderado personalmente el desarrollo de dicho agente y el seteo de sus políticas de seguridad y validación.

Ghosn, Lautaro Gabriel

Entornos y herramientas: VS Code, Notepad++, Neon SGBD, pgAdmin. Despliegues en Render y Vercel.

Modelos de asistencia IAG: Gemini (Google) y ChatGPT (OpenAI).

Aplicación principal: Utilizada fuertemente para agilizar los tiempos de desarrollo y optimizar el rendimiento operativo. Sirvió como apoyo en la configuración de infraestructuras en la nube, consultas sobre arquitectura de microservicios y generación rápida de comandos de despliegue.

Guerrero, Martín

Entornos y herramientas: VS Code, Neon SGBD. Despliegues en Render.

Modelos de asistencia IAG: Claude (Anthropic) y Gemini (Google).

Aplicación principal: Incorporada para optimizar los tiempos de desarrollo y acelerar la codificación. Brindó asistencia en consultas sobre asincronía en redes, apoyo iterativo para la integración y refinación del flujo de procesamiento de lenguaje natural inherente al servicio conversacional del sistema, y soporte sintáctico en Python.

Zielonka, Axel

Entornos y herramientas: VS Code, Neon SGBD. Despliegues en Vercel y Render.

Modelos de asistencia IAG: Claude Code (vía Interfaz de Línea de Comandos - CLI), Claude (Anthropic), ChatGPT (OpenAI) y Gemini (Google).

Aplicación principal: Aplicada estratégicamente para agilizar los tiempos de desarrollo de forma masiva. Permitió el *debugging* ágil de componentes de *React*, facilitó la asistencia directa mediante terminal para la refactorización de estilos, y proveyó soporte generalizado en el maquetado de manuales técnicos para las interfaces de usuario.